Templates and STL in C++

Templates and STL in C++ 1
Introduction 2
Templates 3
Introduction to STL 8
Sequence Containers 8
Associative Containers 9
Container adapters 9
Algorithms 10
Iterators 10
Vector Sequence Container 10
Vectors and Iterators 13

Introduction

The STL is a huge library of small functions that are used all the time for things like searching, moving and copying data structures. First we take a short look at templates, followed by a brief introduction to STL.

Templates

Templates are an important concept when dealing with programming as they allow the programmer to reuse code on many different datatypes. A template can be thought of as a pattern, or mold, which will be reused many times.
With templates, a programmer could create a generic algorithm or class that could operate on many different datatypes.
The principal advantage of templates is that they eliminate the need to have multiple versions of a function or class that each operate on a specific datatype.

Template Functions

A template function is one that can operate on a generic datatype, which suggests that function can be reused with many different types of data. Consider an example where it is desired to create a function that returns the sum of the squares of each number in an array of integers:

```
int sum(int const* pArray, int const ELEMENTS) {
    int sum = 0;

    for (int index = 0; index < ELEMENTS; index++) {
        sum += pArray[index];
    }

    return sum;
}
```

Say later on, it becomes apparent there is a requirement for a function to return the sum of the squares of each number in an array of floats. The above function is rewritten as:

```
float sum(float const* pArray, int const ELEMENTS) {
    float sum = 0;

    for (int index = 0; index < ELEMENTS; index++) {
        sum += pArray[index];
    }

    return sum;
}
```

A problem occurs if there is a change in requirements that dictate a change in the way the function sums the numbers. Every copy of the `sum` function must be modified. This clearly becomes a big inconvenience for complex functions of which there are multiple copies written for specific datatypes. Template functions eliminate this inconvenience as the

programmer can use the same algorithm that is capable of operating on different datatypes. The syntax for a template function is as follows:

```
template <typename T> return-type functionName(parameters) {
      // function body
}
```

where **`template`** is a keyword and **`<typename T>`** is a placeholder for the type of data to be used. Think of `T` as meaning the function will have one generic datatype as a parameter, henceforth referred to as `T`. Now consider an example where we rewrite the above two `sum` functions as one template function:

```
template <typename T>
T sum(T const* pArray, int const ELEMENTS) {
    T sum = 0;

    for (int index = 0; index < ELEMENTS; index++) {
        sum += pArray[index];
    }

    return sum;
}
```

The function now returns an instance of type T and expects to be passed an array, also of type T.
Here is an example of how the function might be used:

```
int main() {

    int const ELEMENTS = 10;
    int intArray[] = {1,2,3,4,5,6,7,8,9,10};

    float floatArray[] =
            {1.1f,2.2f,3.3f,4.4f,5.5f,6.6f,7.7f,8.8f,9.9f,10.1f};

    cout << "Sum of integer array: " <<
             sum(intArray, ELEMENTS) << endl;

    cout << "Sum of float array: " <<
             sum(floatArray, ELEMENTS) << endl;

}
```

In all template functions, C++ will copy the code and replace every instance of the parameterized type name (`T` in this example) with the actual type name. In other words, the compiler performs the drudgery of making multiple copies of the function automatically. In this case, one copy where `T` is replaced with type `int`, and another where `T` is replaced with type `float`.
Note that `T` can be replaced with any name as long as it does not conflict with an existing class or type name. For example, we could use the more descriptive name `DataType`.

Also note that the compiler determines the types of a template function implicitly by analyzing the datatypes that are passed in to the function and creating an appropriate template.

NOTE: The type of a template function is never determined by its return value, only by the parameters.

Template Classes

A template class is similar to a template function, except that a template class is an entire class that operates on a generic datatype. Using the `template` keyword, we can create generic classes to:

```
template <typename T> class ClassName {
     // header for the class
}
```

Then to create a specific instance of this class, use the form

```
ClassName <type> objectName;
```

Consider an example. Say we want to construct a class that is meant to retain a sum and have numerous types of data added to it:

```
template <typename T>
class Adder {
private:
        T m_sum;

public:
       Adder() {
          m_sum = 0;
       }

       // add function.
       void add(T number) {
            m_sum += number;
       }

       // get sum function.
       T sum() {
            return m_sum;
       }
};
```

Here's an example of how the class might be used:

```
int main() {

    Adder<int> intAdder;
    Adder<float> floatAdder;

    float f=0.0f;
    int i;
```

```
        for (i = 0 ; i < 10; i++ )
    {
        intAdder.add(i);
        floatAdder.add(f);
       f += 1.1f;
    }
    cout << "The integer sum using an intAdder: " << intAdder.sum()
         << endl;

    cout << "The float sum using a floatAdder: " << floatAdder.sum()
         << endl;

}
```

Note that the declaration of the template class instance must have its parameterized types explicitly declared, within the angle brackets, after the class name (i.e. `<int>` and `<float>` respectively).
Also note that we are in a class template when writing the member functions for that class.

```
// Adder.h
template <typename T> class Adder {
private:
        T m_sum;

public:
       Adder() {
          m_sum = 0;
       }

       // add function defined elsewhere.
       void add(T number);

       // get sum function defined elsewhere.
       T sum();
};

template <typename T> void Adder<T>::add(T number) {
     m_sum += number;
}

template <typename T> T Adder<T>::sum() {
     return m_sum;
}
```

The `template <typename T>` declaration is still required before each function definition, with the name of the template parameter placed in angle brackets after the template class name.

Multiple parameterized types

Templates do not have to be based on a single generic datatype. A template class or function can have any number of parameterized types. For example:

```
template < typename X, typename Y>
```

Each datatype must be separated by a comma within the angle brackets. In this scenario it is important to use descriptive names for the parameterized types because usually each of these types will have a different purpose.

Consider an example template function that determines the average of an array of arbitrary types:

```
template < typename Sumtype, typename Averagetype>
Averagetype average(Sumtype const* pArray, Averagetype const
ELEMENTS){
     Sumtype sum = 0;

     for (int i = 0; i < ELEMENTS; i++) {
         sum += pArray[i];
     }

     return (Averagetype) sum / ELEMENTS;
}
```

The `SumType` is the datatype to be summed, the `AverageType` is the datatype that will be returned from the function. Note also that `ELEMENTS` is of type `Averagetype` because the average of a list is defined as the sum divided by the number of elements.
The `return` statement assumes it is possible to convert an instance of `Sumtype` into an `Averagetype`. Here's how this function might be used:

```
int main() {

    int intArray[] = {1,2,3,4,5,6,7,8,9,10};

    cout << "average(intArray, 10) = " << average(intArray, 10)
         << endl;
    cout << "average(intArray, 10.0f) = " << average(intArray,
10.0f)
         << endl;

}
```

Note the function invocations: `average(intArray, 10)` and `average(intArray, 10.0f)`. The two functions are called with two different sets of template parameters: `<int, int>` and `<int, float>` respectively.

Introduction to STL

As discussed earlier, the STL defines many general purpose classes and functions that implement popular algorithms and data structures. Most of the STL library uses templates so the classes and functions can be used with any data type offering great flexibility. At the core of the STL are three conceptual items, known as containers, algorithms and iterators.

1. Containers

Containers are objects that hold other objects. Every container class in STL has a certain set of functions. For example, a subset of these functions include:

Function	**Purpose**
Constructor	Constructs the container
Copy Constructor	Copies the contents of the current container into a new container of the same type. Uses a shallow value-copy so that if the container hold pointers, pointers in the new container will point to the same items.
`size`	Returns the number of items in the container
`empty`	Returns true if the container is empty, false if not.
`begin`	Returns an *iterator* (more on this below) pointing to the very first item in the container
`end`	Returns an iterator pointing past the end of the container. This does not point to a valid item.

There are three types of container, each of them support the generic container functions in addition to extra functions defined for their own type.

- Sequence containers
- Associative Containers
- Container adaptors

Sequence Containers

are essentially linear lists and some examples of their implementation in STL are:

The `vector` class – a type of dynamic array, i.e an array that can grow and shrink in size at runtime as needed. A vector defines member functions to manipulate vector elements. This means the vector has all the functionality of the array plus more.

The `list` class – an implementation of a linear linked list.

The `deque` (double ended queue) class – is a vector-like structure that is random access and provides very efficient insertion and removal of objects at either end of the structure. Insertion into the middle of the deque is a much slower operation.

For illustration, two examples of generic sequence functions are:

Function	Purpose
`front`	Returns the first item in the container
`clear`	Totally erases everything in the container

Sequence containers could be used for example in an online, turn-based strategy game to store a group of players that could be cycled through in sequence.

Associative Containers

– This second type of container in STL allows the efficient retrieval of values based on keys.

For example, **the `map` class** provides a mapping of unique keys to values. Values are then retrieved by supplying the key.
.
Example;

mymap[“carlow”]=15000;

cout << mymap[“carlow”];

Container adapters

- These type of containers encapsulate a different class and change the way you access things in it. Examples include:

The `stack` class – a stack facilitates insertion and removal of objects at one end of the structure only according to the LIFO principle (Last In First Out).

The `queue` class – A queue facilites insertion of objects at the rear and removal of objects at the front of the structure according to the FIFO principle (First In First Out).

Both the stack and queue use a deque as their base data structure – but this can be changed.

Finally, there is a miscellaneous category for structures that don't fit under the other container categories. An example of a class that fits this category is:

The `string` class – Technically, the string class is not part of STL. However, strings are containers (contain an array of characters) objects and it is possible to use appropriate STL algorithms on them. The string class comes complete with member functions (such as `find()`, `substr()` and so on) and overloaded operators (such as + for string concatenation).

Algorithms

The STL library provides a collection of generic algorithms defined in **<**`algorithm`**>** which can be applied to a variety of different data structures. STL comes with more than 70 algorithms, ranging from very simple to the extremely complex. Examples include `min(), max(), search(), sort()` to name a few. Most STL algorithms operate on a sequence of elements, and the algorithms access data via the use of iterators. We will look at an example of an STL algorithm after discussing iterators.

Iterators

An iterator is a kind of generalised pointer that may be used to traverse the elements of a container class. At any point in time an iterator is positioned at exactly one place in a collection, and will remain there until moved. Iterator syntax uses the ++, --, and * operators which are familiar to users of pointers.

Vector Sequence Container

The vector sequence container has been designed to replace most applications of arrays. As new data is added to containers such as vectors (and indeed strings), they automatically grow to accommodate the new insertions.

Let's start with a simple example that illustrates some of the basic member functions of the vector class:

```
#include <vector>
#include <iostream>
```

```
using namespace std;

int main() {
   vector<int> v;          // declare an int vector

   v.push_back(10);   // add 10 to the end of the vector
   v.push_back(20);   // vector automatically resizes
   v.push_back(30);   // whenever you add items to it

   v.at(0) = v.at(2); // copy 30 into 10

   v[0] = v[2];       // same as v.at(0) = v.at(2)

   v.pop_back();      // erase the last item
   int size = v.size();     // size should be 2
   v.resize(32);      // make vector hold 32 items
                      // cells from subscript 2 to subscript 31 are
                        // padded with zeros

}
```

Vectors have a capacity which tell us how many elements the container can hold in the memory it has already allocated. The size of a vector is the actual number of elements that are currently in the container. Note that (capacity - size) tell us how much unoccupied memory a vector (or string) has. If (capacity - size) is zero, then there is no spare capacity and a further insertion into the container will cause the container to dynamically allocate more memory.

Here's another example to illustrate:

```
#include <vector>
#include <iostream>

using namespace std;

int main() {
   vector<int> v(10,0);     // declare an int vector of size 10, all
                          // elements are initialized to zero

   cout << v.capacity() << endl;
                      // capacity is 10 (memory reallocation will
                        // occur if another element is added)

   cout << v.size() << endl;     // size is 10


   v.push_back(20);   // memory is reallocated to accommodate growth
                      // capacity increases to 15, size is 11

}
```

This memory reallocation process has **three** stages:

1. Allocate a new block of memory that is some multiple of the container's current capacity. In most implementations, vector and string capacities grow by a factor of two each time, i.e. their capacity is doubled each time the container must be expanded.

2. Copy all the elements from the container's old memory into its new memory.

3. Deallocate the memory of the old objects.

Clearly, these three steps cause a performance hit. However, vector memory reallocation might not occur at a performance-critical part of your program, in which case it may not be relevant. Or perhaps the vector has a small number of elements, so the reallocation cost might be insignificant.

However, you don't want to perform them any more frequently than you have to. One way of controlling dynamic memory reallocation is to use the reserve member function which sets the capacity of the vector. More specifically, **reserve(*n*)** forces the container to change its capacity to at least *n* given that *n* is no less than the current size. This typically forces a reallocation because the capacity needs to be increased. If *n* is less than the current capacity, vector ignores the call and does nothing.
The key to avoiding reallocations is to use reserve to set a container's capacity to a sufficiently large value as soon as possible, ideally right after the container is constructed.
Let’s consider another example:

```
#include <vector>
#include <iostream>

using namespace std;

int main() {
   vector<int> v(10,0);     // declare an int vector of size 10, all
                          // elements are initialized to zero

   cout << v.capacity() << endl;
                        // capacity is 10 (memory reallocation will
                          // occur if another element is added)

   cout << v.size() << endl;     // size is 10

   v.reserve(20);           // reserve memory for 10 additional
                            elements
                        // size is 10, capacity is now 20

   v.push_back(12);
}
```

What if we don't know how much space we will eventually require in the vector? The solution is to reserve more space than you think you will ever need, after which you can subsequently trim the excess capacity from the vector. Say a vector called products currently holds 90,000 `integers` with a capacity of 100,000. At some point most of its contents are erased down to just 10 integers , however its capacity remains stuck at 100,000. There is a simple technique to reduce the capacity to the amount it currently needs, and this capacity reduction is commonly referred to as "shrink-to-fit". To shrink our vector Products:

```
vector<int>(products).swap(products);
```

or a simpler.

```
products.shrink_to_fit();
```

Exercise :

Create the products vector with capacity 100,000 and fill the products array with 90,000 integer. Print out the size and capacity. Resize the vector to size of 10. Print out the size and capacity. Add the line of code above. Print out the size and capacity.

Extra TIP: Another related and useful feature of swap, is to quickly clear a vector of its contents. For example:

```
vector<int>().swap(products);
```

swaps the contents of `products` with an empty vector.

Vectors and Iterators

Let's now consider an example that illustrates how to use iterators with the vector class:

```
#include <vector>
#include <algorithm>
#include <iostream>


using namespace std;

int main() {

	vector<int> v;
```

```
	v.push_back(30);
	v.push_back(10);
	v.push_back(5);		// order is 30,10,5

	// get an iterator that points to the first element in v
	vector<int>::iterator iter = v.begin();

	int i = *iter;			// get value that iterator points to (30)
	++iter;				// move to the next item (10)
	i = *iter;		// get that item (i now = 10)
	--iter;				// move back to the first item
	*iter = 35;		// order is now 35,10,5


	v.insert(iter, 3, 100); // insert 3 ints of value 100 at the
	// first index (given by iter)
	// order is now 100,100,100,35,10,5
	iter = v.begin();
	iter++;

	v.erase(iter);			// remove the element pointed to by iter

	iter = v.begin();
	v.erase(iter, iter + 2); // remove the next two elements, order

	// sort is an STL algorithm
	sort(v.begin(), v.end());	// sort the vector

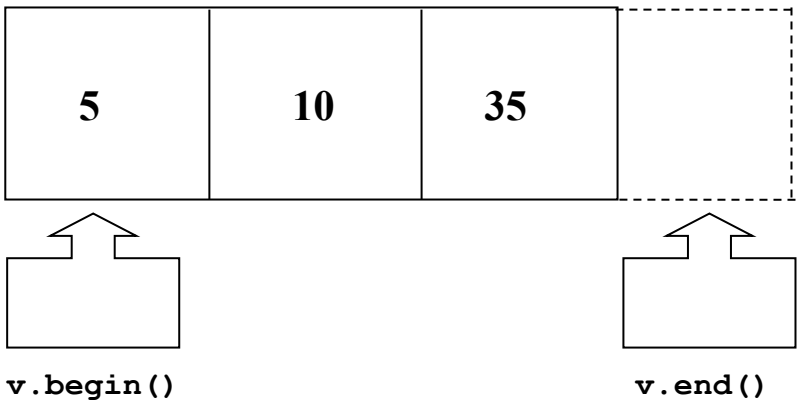


	// iterate over the entire vector v and output the sorted vector
	for (iter = v.begin(); iter != v.end(); iter++) {
		cout << *iter << endl;
	}

}
```

An iterator acts exactly like it were a pointer into an array, so you can move it forward and backward, and dereference it. Note the line in main where the function `sort()` is invoked. The `sort()` algorithm is an STL algorithm that takes two random access iterators and sorts everything between them, not including the item that the end iterator points to. This is an important point. The `end()` member function returns an iterator one past the last element in the container. This means the for loop will continue until `iter` has moved through all the

elements of the vector `v`. Consider an abstract view of the iterator returned by a call to the member functions `begin()` and `end()` in the above for loop:



The `end()` vector member function returns an iterator that's one past the last element in the vector, not the *last* element. Therefore, you can't get a value from the iterator returned by `end()`. This might seem counterintuitive, but it works well for loops that move through a container.

Some general points regarding STL containers (including vectors)

1) Objects are copied into containers and copied from containers

STL containers, such as vectors, would not be very useful if we could just store primitive types in them. All STL containers can store both primitive types and objects. The following example demonstrates how we can store instances of a class `Landmark` in a vector:

```
#include <vector>
using namespace std;

class Landmark {
public:
	int m_x;
	int m_y;

public:
	Landmark (int x, int y)
	{
		m_x=x;
		m_y=y;
	}


	};



int main()
{
```

```
    vector<Landmark> LandmarkContainer;

  //calls Landmark normal constructor that take 2 parameter
    Landmark carlow(0,0);

    LandmarkContainer.push_back(carlow);

    LandmarkContainer[0].m_x=10;

}
```

Since monster is copied into the container, altering the object in position 0 in the vector does not alter the monster object.

Note that when we add an object to the vector, it gets copied into the vector by the push_back method. Likewise, if we were to request an object from the vector, a copy would be returned. In general terms, STL containers always copy in and copy out. Once an object is in a container, it is not uncommon for it to be copied further. Containers such as vectors, strings or deques will typically move (copy) elements around if a new element is inserted or erased from that container. l

If you fill a container with objects where copying is expensive, the act of putting the objects into the container could prove to be a performance bottleneck.

2) Always use a vector where you might normally have used a static array

Unlike arrays, vectors know their size, so there is no need to pass the number of elements as an additional argument to a function expecting an array. There is very little extra memory consumption associated with using vectors (in comparison to arrays)

Consider the following array declaration:

```
SomeClass someArray[100];
```

This populates someArray with 100 objects of type SomeClass, i.e. 100 constructor functions are executed! Now consider:

```
vector<SomeClass> someVec;
someVec.reserve(100);
```

No constructors are executed, the container is ready to store 100 objects.

3) Do not use a vector if your application needs to frequently insert and remove elements from the middle of the container. Use a list instead.

4) Recall the above point in relation to performance:

“If you fill a container with objects where copying is expensive, the act of putting the objects into the container could prove to be a performance bottleneck”

It is possible to store pointers to objects in a container. If you allocate heap memory and store pointers to that heap memory in a container (e.g. vector), the container will NOT free that memory when the container is destroyed. Deleting memory is the responsibility of the programmer, not the container.

Consider a trivial example to illustrate this point:

```
class SomeClass {

public:

      ~SomeClass() {
            cout << "Object destroyed: " << endl;
      }

};


void populate() {
      vector<SomeClass *> vec;
      SomeClass *g = new SomeClass();
      vec.push_back( g );

}
```

The destructor for object SomeClass is never called when vec goes out of scope.

5) Do not create the container until you are sure it is going to be used.

Containers such as vectors allocate (a small) amount of memory upon construction – this can be a wasted construction if the container is not used.

Consider an example:

```
bool populate(SomeClass *ptr) {
      Vector<int> vec;

      if ( ptr == 0 ) {
            return false;
      }
      // Else do something with vec here

}
```

In this example, vec is created but may never be used. Where should we declare vec?

When to use vectors:

Always use wherever you might use a static array (i.e. arrays created on the stack). Unlike arrays, vectors know their size, so there is no need to pass the number of elements as an additional argument to a function expecting an array.
When not to use vectors: If your application has a requirement to insert and remove elements from the middle of a sequence, do not use a vector, use a list instead. For example, removing an element from the middle of the vector, results in all subsequent elements being copied back one position to 'fill the gap'.

Iterator, pointer, and reference invalidation

Note carefully that in general, insertions or erasures on contiguous-memory containers (such as vectors or strings) may invalidate all iterators, pointers, and references into the container.

Let's illustrate with a simple example:

```
vector<int> v;
v.push_back(30);
// Get an iterator that points to the first element in v
vector<int>::iterator iter = v.begin();
v.push_back(10); // insertion: iter no longer valid

cout << *iter << endl;      // runtime error! iter not valid
```

We add number 30 to the vector, then request an iterator to the first element, then add another number 10 to the vector. This latter insertion causes the invalidation of the previously requested iterator which is now no longer safe to use. Similarly, we can observe similar behaviour with the deletion of elements from the vector. To delete from a vector, we can use the `erase()` member function which has several flavours. The version used below expects an iterator as an argument – this tells erase what element to delete from the vector:

```
vector<int> v;
v.push_back(30);
v.push_back(10);
// get an iterator that points to the first element in v
vector<int>::iterator iter = v.begin();
v.erase(iter);  // delete 30, iter now invalidated.
iter++;         // runtime error! iter is invalid.
```

Invoking erase not only invalidates all iterators pointing to the erased element, it also invalidates all iterators beyond the erased element. Consider an example:

```
vector<int> v;
v.push_back(30);
v.push_back(20);
v.push_back(10);
vector<int>::iterator iter1 = v.begin();    // point to 30
vector<int>::iterator iter2 = --(v.end()); // point to 10
vector<int>::iterator iter3 =  --iter2;     // point to 20
```

```
v.erase(iter3);// delete 20, iter3 and iter 2 now invalidated, iter1
ok
```

Furthermore, the invalidation of iterators, pointers, and references also occurs whenever the three steps association with container memory reallocation occur. Consider an example:

```
vector<int> v;
v.push_back(30);
v.push_back(20);
vector<int>::iterator iter1 = v.begin();    // point to 30
v.reserve(10); // iter1 no longer valid.
```

erase-remove

For contiguous memory containers (vectors, deqeues, strings), the best approach to removing all objects with a specific value is the erase-remove idiom.
The remove part uses the STL `std::remove<>()` function:

```
ForwardIterator remove(ForwardIterator start,ForwardIterator finish,
        const T& value);
```

Let's see how remove works with an example:

```
(3,0,1,2,3,4,5,6,7,8,9)
```

```
#include <algorithm>  // required for std::remove below

vector<int> v;

v.push_back(3);
for (int i = 0; i < 10; ++i) {
     v.push_back(i);  // Note: 3 will now appear twice in the vector
}

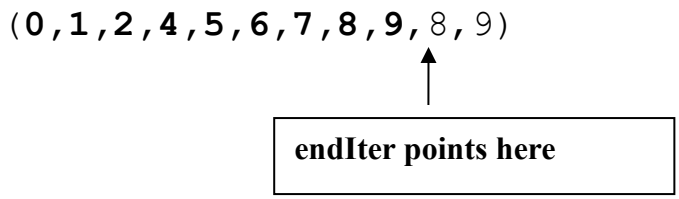
vector<int>::iterator endIter = remove( v.begin(), v.end(), 3 );
```

After calling remove, the contents of the vector look like:

```
 (0,1,2,4,5,6,7,8,9,8,9)
```

remove has deleted all occurrences of the number 3 (as we might expect), but the vector still has 11 elements (as we might not expect). As each occurrence of 3 is removed, each subsequent element is **copied** down one position in the vector. The return value from remove,

an iterator, is effectively an end iterator that points one past the end of our new sequence of numbers, i.e.:



So now we must call the erase function to delete the two 'junk' values from the end of our vector:

```
#include <algorithm>  // required for std::remove below

vector<int> v;

v.push_back(3);
for (int i = 0; i < 10; ++i) {
     v.push_back(i);  // Note: 3 will now appear twice in the vector
}

vector<int>::iterator endofValidElementsIter = remove( v.begin(),
v.end(), 3 );

v.erase(endofValidElementsIter, v.end() );
```

Our vector now looks like:
(0,1,2,4,5,6,7,8,9)

i.e. 8 elements as expected.

Exercise: Can you work out how the remove, erase operations can be combined into one line?

Erasing specific objects from containers

There may be many occasions when we want to get rid of every object in a container for which certain conditions hold. Consider an example: say we have a vector of type `Particle`. Every frame, we iterate through the vector of particles and draw them. Additionally, each particle has a time to live (`m_timetolive`) that is decremented every frame. When the `m_timetolive` expires, we want to remove that particle from the vector so it doesn't get drawn. We can achieve this using *function predicates*.

A **predicate** is a C++ function that returns true or false.

```
// Sample Particle class.
class Particle {
private:
      int m_timetolive;

public:
      Particle(int ttl) : m_timetolive (ttl) {}

      int timetolive (){ return m_timetolive; }
};

// A global predicate.

bool isExpired(Particle & particle) {

      // Assumes ttl can never be negative.

      return (particle.timetolive () == 0);
}

int main() {
   vector<Particle> v;
   for (int i = 0; i < 10; ++i ) {
      // Each Particle has a different timetolive
      v.push_back( Particle(i) );
   }

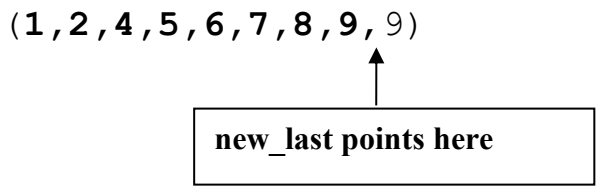

   // Lets erase the particles with a timetolive of 0.

   v.erase(remove_if(v.begin(), v.end(), isExpired), v.end());
 }
```

Before calling `v.erase()`, the vector holds 10 Particle instances whose ttl values are: `(0,1,2,4,5,6,7,8,9)`

The call to `v.erase()` uses `remove_if`, a generic STL algorithm that removes from the range [`v.begin()` to `v.end()`] every element such that the predicate `isExpired()` is true.

Note that the `remove_if` algorithm takes a **function pointer** to the `isExpired()` predicate. `remove_if` returns an iterator `new_last` such that the range `[first, new_last]` contains no elements for which `pred` is `true`, i.e.



All that remains is to remove the range from `new_last` to `v.end()`, i.e.

```
v.erase(remove_if(v.begin(), v.end(), isExpired), v.end());
```

In summary, predicates are widely used in the STL. Predicate functions are commonly passed as parameters to algorithms like remove_if and the various sorting algorithms.

Range member functions

A range member function is a container member function that, like STL algorithms, uses two iterator parameters to specify a range of elements over which something should be done. To consider why range member functions are incredibly useful, let's illustrate with a simple problem:

Copy an existing array of ints into the front of a vector.

A first attempt might use a loop with the single-element version of `insert()`:

```
vector<int> v;

int data[] = { 32, 71, 12, 45, 26, 80, 53, 33 };
int numValues = 8;
vector<int>::iterator insertLoc = v.begin();


for (int i = 0; i < numValues; ++i) {
	insertLoc = v.insert(insertLoc, data[numValues-i-1]);
}
```

A 2nd attempt uses a range member function:

```
v.insert(v.begin(), data, data + numValues);
```

`insert` is both a member function of `std::vector` and also a function that works with iterator ranges (hence a range member function). Why is this approach preferable to our first attempt above that uses an explicit loop? There are three reasons:

1) The single-element `insert` method is called repeatedly, i.e. `numValues` times. Using the range form of `insert`, there is only one function call, a savings of `numValues - 1` calls.

2) Each time `insert` is called to add a new value to `v`, every element above the insertion point must be moved up one position to make room for the new element. So the element at position *p* must be moved up to position *p+1*, etc.

Here, we are inserting `numValues` elements at the front of `v`. Therefore, each element in `v` prior to the insertions will have to be shifted up a total of `numValues` positions.

Each element will be shifted up only one position each time `insert` is called, so each element will be moved a total of `numValues` times.

In the general case, inserting `numValues` new objects one at a time into the front of a `vector<Particle>` holding `n` elements exacts a cost of `n*numValues` function calls.

The C++ Standard requires that range insert functions move existing container elements directly into their final positions at a cost of one move per element. The total cost is `n` moves, `numValues` to the copy constructor for the type of objects in the container. Compared to the single-element insert strategy which performs `n*(numValues)` moves.

3) The third reason to use a range member function relates to memory allocation. When you try to insert an element into a vector whose memory is full, the vector allocates new memory with more capacity, copies its elements from the old memory to the new memory, destroys the elements in the old memory, and deallocates the old memory. Then it adds the element that is being inserted. It is possible that some vector implementations will perform 10 new memory allocations if 1000 new elements are inserted one at a time.
A range insertion can figure out how much new memory it needs before it starts inserting things (assuming it is given forward iterators), so it need not reallocate a vector's underlying memory more than once.

Sorting a vector

```
#include <vector>
#include <algorithm>
#include <iostream>
using namespace std;


class MyClass{
public:
    int num;
};
```

```
// Compares two myClasses according to num.
bool Compare(MyClass &myClass1, MyClass& myClass2)
{
    return (myClass1.num < myClass2.num);
}

int main()
{
    vector<MyClass> v{ { 6}, { 1 }, { 2 }, { 4 } };


    sort(v.begin(), v.end(), Compare);


    for (int i = 0; i < v.size(); i++)
    {
        cout <<  v[i].num << endl;

    }return 0;
}
```